فصل ۴: بازگشت و ردگیری فراخوانی

چرا این فصل؟ تست‌های بازگشت دو نوع‌اند: «خروجی چیست؟» (ترتیب چاپ قبل یا بعد از فراخوانی) و «چند بار فراخ
می‌شود؟». هر دو با یک روش حل می‌شوند: **درخت فراخوانی بکش.** این فصل پایهٔ روابط بازگشتی (ساختمان داده فصل
برنامه‌نویسی پویا هم هست. **اعتبار:** خروجی همهٔ کدها با `gcc` اجرا و چک شده است. **زمان مطالعه:** یک بلوک ۹۰ دقیقه‌ا

درس ۱: ساختار تابع بازگشتی و ترتیب چاپ

هر تابع بازگشتی دو بخش دارد: **حالت پایه** (توقف) و **فراخوانی روی مسئلهٔ کوچک‌تر**. هر فراخوانی قاب مخصوص خودش
پشتهٔ فراخوانی دارد، با متغیرهای محلی جداگانه.

قاعدهٔ طلایی ترتیب چاپ:

- چاپ **قبل** از فراخوانی ⇐ در مسیر **رفت** چاپ می‌شود (ترتیب نزولی).
- چاپ **بعد** از فراخوانی ⇐ در مسیر **برگشت** چاپ می‌شود (ترتیب صعودی).

```
d p(int n) { if (n == 0) return; printf("%d ", n); p(n-1); }   // p(3): 3 2 1
d q(int n) { if (n == 0) return; q(n-1); printf("%d ", n); }   // q(3): 1 2 3
d r(int n) { if (n <= 0) return; printf("%d ", n); r(n-2); printf("%d ", n); }
r(5): 5 3 1 1 3 5   (یک بار در رفت، یک بار در برگشت)
```

تمرین درس ۱

```
1
 g(int n) { if (n == 0) return 0; return n % 10 + g(n / 10); }      // g(1234)?
2
d b(int n) { if (n > 1) b(n / 2); printf("%d", n % 2); }            // b(13)?
```

🔑 **جواب‌ها:** پاسخ‌نامهٔ انتهای فصل، بخش ۱ (درس ۱: ساختار تابع بازگشتی و ترتیب چاپ)

درس ۲: شمردن فراخوانی‌ها و توابع کلاسیک

تابع	تعداد فراخوانی یا هزینه
فاکتوریل `f(n) = n·f(n−1)`	n فراخوانی، `O(n)`
فیبوناچی ساده `fib(n) = fib(n−1) + fib(n−2)`	`2·F(n+1) − 1` فراخوانی (نمایی). مثلاً `fib(5)`: ۱۵ و `fib(6)`: ۵
هانوی	$2^n - 1$ حرکت
ب.م.م اقلیدسی `gcd(b, a % b)`	`O(log min(a,b))`
توان سریع `t = pw(x, n/2)` و بعد `t·t`	`O(log n)` ضرب

تابع	تعداد فراخوانی یا هزینه
`h(n) = h(n/2) + h(n/2)` (دو فراخوانی **جدا**)	`2n − 1` فراخوانی برای n توان ۲

روش شمردن: رابطهٔ «تعداد فراخوانی» را بنویس: `C(n) = 1 + C(زیرمسئله‌ها)`. مثلاً برای فیبوناچی `C(n) = 1 + C(n−1) + C(n−2)` و `C(0) = C(1) = 1`.

⚠️ `pw(x, n/2) * pw(x, n/2)` (دو بار صدا زدن) با `t = pw(x, n/2); t * t` (یک بار) فرق دارد. اولی `O(n)` و دومی `O(log n)`.

متغیر static در بازگشت

یک متغیر `static` بین **همهٔ** فراخوانی‌ها مشترک است، نه اینکه هر قاب نسخهٔ خودش را داشته باشد.

📝 آزمون فصل ۴

این درس در کنکور ارشد کامپیوتر سؤال سابقه ندارد. پس این ۱۰ تست **تألیفی** هستند و خروجی همه با `gcc` تأیید ش است.

زمان: ۱۵ دقیقه.

۱. `int f(int n) { if (n <= 0) return 0; return n + f(n - 2); }` مقدار `f(7)`؟ ۱) ۱۶ ۲) ۲۸ ۳) ۱۲ ۴)

۲. خروجی `p(3)`؟

```
d p(int n) { if (n > 0) { p(n - 1); printf("%d", n); p(n - 1); } }
```

۱) `123` ۲) `1213121` ۳) `3212123` ۴) `1121312`

۳. مقدار برگشتی و تعداد کل فراخوانی‌های `fib(6)`؟

```
 c = 0;
fib(int n) { c++; if (n < 2) return n; return fib(n-1) + fib(n-2); }
```

۱) `25 8` ۲) `15 8` ۳) `25 13` ۴) `13 8`

۴.

```
 g(int x) { if (x > 100) return x - 10; return g(g(x + 11)); }
```

مقدار `g(95)`؟ ۱) ۹۵ ۲) ۹۱ ۳) ۱۰۱ ۴) ۸۵

۵.

```
 s(int n) { static int k = 0; k += n; if (n > 1) s(n - 1); return k; }
```

مقدار `s(4)`؟ ۱) ۴ ۲) ۷ ۳) ۱۰ ۴) ۲۴

۶.

```
 f(int n) { if (n == 1) return 1; if (n % 2 == 0) return f(n / 2); return f(3*n + 1) + 1; }
```

مقدار `f(6)`؟ ۱) ۲ ۲) ۳ ۳) ۴ ۴) ۸

۷. خروجی `r("abc")`؟

```
d r(const char *s) { if (*s) { r(s + 1); putchar(*s); } }
```

۱) `abc` ۲) `cba` ۳) `a` ۴) `c`

۸.

```
 m(int a, int b) {
if (b == 0) return 0;
if (b % 2 == 0) return m(a + a, b / 2);
return m(a + a, b / 2) + a;
```

مقدار `m(7, 6)`؟ ۱) ۱۳ ۲) ۴۲ ۳) ۴۹ ۴) ۲۸

۹. مقدار برگشتی `h(8)` و تعداد کل فراخوانی‌ها؟

```
 c = 0;
 h(int n) { c++; if (n <= 1) return 1; return h(n/2) + h(n/2); }
```

۱) `15 8` ۲) `7 4` ۳) `8 8` ۴) `4 1`

۱۰.

```
 rev(int n, int acc) { return n == 0 ? acc : rev(n / 10, acc * 10 + n % 10); }
```

مقدار `rev(1203, 0)`؟ ۱) ۳۰۲۱ ۲) ۳۲۱ ۳) ۱۲۰۳ ۴) ۲۱۰۳

> 🔑 **جواب‌ها و راه‌حل:** پاسخ‌نامهٔ انتهای فصل، بخش ۲ (آزمون فصل ۴)

🗂 کارت خلاصهٔ فصل ۴

- چاپ قبل از فراخوانی: رفت (نزولی). چاپ بعد از فراخوانی: برگشت (صعودی). هر دو: الگوی آینه‌ای.
- تعداد فراخوانی را با `C(n) = 1 + Σ C(زیرمسئله)` بشمار. فیبوناچی: `2F(n+1) − 1` .
- دو بار صدا زدن زیرمسئلهٔ یکسان، هزینه را چند برابر می‌کند. یک بار صدا زدن و ذخیره کردن، `O(log n)` می‌دهد.
- static بین همهٔ قاب‌ها مشترک است. تابع ۹۱: همیشه ۹۱ برای `x ≤ 101` .

🔑 پاسخ‌نامهٔ فصل

بخش ۱: درس ۱: ساختار تابع بازگشتی و ترتیب چاپ

۱. `10` (جمع ارقام)

۲. `1101` . نمایش دودویی ۱۳ است: رقم‌ها در برگشت چاپ می‌شوند، پس پرارزش‌ترین رقم اول می‌آید.

بخش ۲: آزمون فصل ۴

سؤال	۱	۲	۳	۴	۵	۶	۷	۸	۹	۱۰
جواب	۱	۲	۱	۲	۳	۳	۲	۲	۱	۱

۱. `7 + 5 + 3 + 1 = 16`

۲. `p(1) = "1"` ، `p(2) = p(1) 2 p(1) = "121"` ، `p(3) = "121" 3 "121" = 1213121`

۳. `fib(6) = 8` و تعداد فراخوانی `2·F(7) − 1 = 2·13 − 1 = 25`

۴. تابع ۹۱ مک‌کارتی: برای هر `x ≤ 101` جواب **۹۱** است.

۵. k مشترک است: `4 + 3 + 2 + 1 = 10` . همهٔ قاب‌ها همان k را برمی‌گردانند.

۶. `f(6) = f(3)` ، `f(3) = f(10) + 1` ، `f(10) = f(5)` ، `f(5) = f(16) + 1` ، و `= f(8) = f(4) = f(2) = f(1) = 1`. جمع `1 + 1 + 1 = 3` .

۷. چاپ بعد از فراخوانی است، پس رشته وارون می‌شود: `cba` .

۸. ضرب دهقان روسی: `m(a,b) = a·b` ، یعنی `7 × 6 = 42` .

۹. مقدار `h(8) = 8` . تعداد فراخوانی: `C(1) = 1` ، `C(2) = 3` ، `C(4) = 7` ، `C(8) = 15` .

۱۰. ارقام وارون می‌شوند: `3021` .